ZERO TO SENIOR ENGINEER · LAUGHTALE.CO.ZA

MY TECH STACK

WHY I CHOSE IT

5	1	5
Layers	System Rule	Q&A Preps

01 · THE STACK

02 · IF THEY PUSH BACK

- Each row: LAYER → WHY I CHOSE IT → WHY NOT THE ALTERNATIVE → KEY TERMS
- Gold section = interview Q&A prep — rehearsed answers to common pushback

Print: Chrome/Edge → Print → A4 · Background graphics ON · Landscape

DOMAIN 01

THE STACK

Each language does exactly one job it's uniquely good at, nothing more. That's a deliberate choice, not a default.

5
LAYERS

LAYER	WHY I CHOSE IT	WHY NOT THE ALTERNATIVE	KEY TERMS
PYTHON THE BRAIN — ORCHESTRATION & AI	Every major AI SDK (Anthropic, OpenAI) is Python-first. Best-in-class libraries for data validation (Pydantic), async I/O (httpx/asyncio), and ETL — it's the layer that talks to everything else.	Not Node.js — Python's data and AI ecosystem is deeper and more mature for orchestration work.	*Pydantic, httpx, asyncio, orchestration*
C# / .NET THE BRAWN — PERFORMANCE & ENTERPRISE LOGIC	The CLR gives explicit control over memory (Span<T>, stack vs heap) and predictable GC behaviour, essential when a service must stay fast under real load. Strong typing and DI keep long-running business logic safe to extend.	Not Python here — the GIL and dynamic typing make it the wrong tool once performance and predictability actually matter.	*Span<T>, CLR, DI, GC*
POWERSHELL THE GLUE — INFRASTRUCTURE & HARDENING	A .NET object pipeline, not text parsing — you filter live typed objects (DateTime, Process) directly. Native to Windows hardening, security auditing, and idempotent infrastructure (DSC).	Not Bash — Bash parses fragile text output; PowerShell returns typed objects, safer at scale.	*DSC, object pipeline, hardening*
POSTGRESQL THE MEMORY — PERSISTENCE & INTEGRITY	ACID transactions and constraints guarantee data integrity. JSONB adds document flexibility without losing relational structure. Triggers enforce audit logging the application layer can't bypass.	Not MongoDB — this data is relational by nature (jobs, users, transactions) and needs strict consistency, not eventual consistency.	*ACID, JSONB, triggers*
TYPESCRIPT THE FACE — UI & CONTRACT ENFORCEMENT	Compile-time types catch a class of bugs before they ship. Paired with Zod for runtime validation, the Python⇔TypeScript boundary becomes an enforced contract — a schema mismatch fails loudly, not silently.	Not plain JavaScript — JS has no compile-time safety net, so backend changes surface as silent undefined bugs instead of caught errors.	*Zod, compile-time types, contract*

THE SYSTEM RULE

PYTHON ORCHESTRATES. C# PERFORMS. POWERSHELL MANAGES. POSTGRESQL PERSISTS. TYPESCRIPT DISPLAYS. NO LAYER DOES ANOTHER LAYER'S JOB.

Flow: TypeScript (UI) → Python (API + orchestration) → PostgreSQL (data) . C# handles performance-critical services Python calls into. PowerShell provisions and hardens the infrastructure underneath all of it.

DOMAIN 02

IF THEY PUSH BACK

Rehearsed answers to the five questions a five-language stack always invites.

5
QUESTIONS

Q#	QUESTION	ANSWER	THEME
Q1	Why five languages instead of one?	No single language is best at everything. One language everywhere means constantly fighting its weaknesses, or quietly accepting bad trade-offs. I match the tool to the job.	*philosophy*
Q2	Isn't five languages more to maintain?	The complexity is bounded — each layer has one job and a defined contract with its neighbours (e.g. Zod validates the Python→TypeScript boundary). That's cheaper than using the wrong tool for a performance- or integrity-critical job.	*maintainability*
Q3	Where would you cut this down for a smaller project?	Python + PostgreSQL + TypeScript ships most products end to end. C# only enters once I hit a measured performance ceiling Python can't solve. PowerShell is Windows-specific, irrelevant on Linux containers.	*scalability*
Q4	How do you stop layers from doing each other's job?	Strict, one-directional data flow. TypeScript only talks to Python's API, never the database directly. Every boundary has an enforced schema, so violations fail loudly instead of silently.	*architecture*
Q5	What's the single biggest risk in a stack like this?	Contract drift between layers. I mitigate it by making the schema the source of truth (Pydantic/Zod) rather than trusting documentation or memory to stay in sync.	*risk*